

RBE 502 Project Report - Quadrotor Controllers

Pranay Katyal

December 13, 2024

Abstract

This project focuses on designing and implementing controllers for a quadrotor (CrazyFlie2.0) to track trajectories in a restricted airspace. A Proportional-Derivative (PD) controller and a Linear Quadratic Regulator (LQR) were implemented using MATLAB. Their performances were compared for trajectory tracking tasks using MATLAB simulations.

1 Introduction

The objective of this project is to design controllers to achieve stable trajectory tracking for a quadrotor. A PD controller is used for simplicity and ease of implementation, while LQR is employed for optimal performance. The quadrotor dynamics and control were simulated using MATLAB.

2 Quadrotor Dynamics

The quadrotor dynamics are governed by Newton's equations for translational motion and Euler's equations for rotational motion. The state equations are linearized around the hover condition to simplify controller design. The key equations are:

$$\dot{x} = Ax + Bu, \tag{1}$$

where x represents the state vector and u the control inputs.

3 Control Design

3.1 PD Controller

The PD controller is designed to regulate both position and attitude of the quadrotor. The control law is given as:

$$u = K_p e + K_d \dot{e}, \quad (2)$$

where e is the error, K_p is the proportional gain, and K_d is the derivative gain.

3.2 LQR Controller

The infinite horizon LQR controller minimizes a quadratic cost function:

$$J = \int_0^\infty (x^T Q x + u^T R u) dt, \quad (3)$$

where Q and R are weighting matrices, linked to the weight of states and control inputs respectively. The optimal control law is:

$$u = -Kx, \quad (4)$$

with K derived from the solution of the algebraic Riccati equation.

4 Simulation Results

Simulations were conducted with two trajectories: **a circle and a diamond helix**. These trajectories were given by the trajectory planner. The tracking performances of the PD and LQR controllers were compared. The following MATLAB codes were used to implement the controllers.

4.1 PD Controller MATLAB Code

```
function [F, M, trpy, drpy] = pid_controller(qd, t, qn, params) % pranay katyal
% CONTROLLER quadrotor controller
% The current states are:
% qd{qn}.pos, qd{qn}.vel, qd{qn}.euler = [roll;pitch;yaw], qd{qn}.omega
% The desired states are:
% qd{qn}.pos_des, qd{qn}.vel_des, qd{qn}.acc_des, qd{qn}.yaw_des, qd{qn}.yawdot_
% Using these current and desired states, you have to compute the desired contro
```

```

% ===== Your code goes here =====

persistent gd;
persistent icnt;
if isempty(gd)
    gd = zeros(0,3);
    icnt = 0;
end
icnt = icnt + 1;

% ===== Your code starts here =====
%% Parameter Initialization

% Controller gains for position and attitude
Kp_pos = [5; 5; 8];          % Position control gains
Kd_pos = [15; 15; 5];        % Velocity control gains
Kp_att = [205; 205; 205];    % Roll, pitch, yaw control gains
Kd_att = [50; 50; 50];      % Angular velocity control gains
%Parameters :

m = params.mass;
g = params.grav;

% Quadrotor desired trajectory values
pos_des = qd{qn}.pos_des;
vel_des = qd{qn}.vel_des;
acc_des = qd{qn}.acc_des;
yaw_des = qd{qn}.yaw_des;
yawdot_des = qd{qn}.yawdot_des;
% Quadrotor current states values
pos = qd{qn}.pos;
vel = qd{qn}.vel;
euler = qd{qn}.euler;        % Roll, pitch, yaw angles
omega = qd{qn}.omega;        % Angular velocities [p, q, r]

% Position Control for Thrust
pos_error = pos_des - pos;
vel_error = vel_des - vel;
acc_com = acc_des + Kp_pos .* pos_error + Kd_pos .* vel_error;

```

```

% Compute thrust F
F = m * (g + acc_com(3));

% Compute Desired Roll, Pitch, Yaw for Position Control
phi_des = (1 / g) * (acc_com(1) * sin(yaw_des) - acc_com(2) * cos(yaw_des));
theta_des = (1 / g) * (acc_com(1) * cos(yaw_des) + acc_com(2) * sin(yaw_des));
psi_des = yaw_des;

%Attitude Control for Moments (Roll, Pitch, Yaw)
euler_des = [phi_des; theta_des; psi_des];
euler_error = euler_des - euler;
omega_des = [0; 0; yawdot_des]; % Given zero desired rates for roll and pitch
omega_error = omega_des - omega;

% Compute moments M
M = params.I * (Kp_att .* euler_error + Kd_att .* omega_error);

% ===== Your code ends here =====

%Output trpy and drpy as in hardware
trpy = [0, 0, 0, 0];
drpy = [0, 0, 0, 0];

end

```

4.2 LQR Controller MATLAB Code

```

% CONTROLLER quadrotor controller using LQR PranayKatyal
function [F, M, trpy, drpy] = lqr_controller(qd, t, qn, params, trajhandle)

% Persistent variables for system matrices
persistent A B Q R K;

% Initialize system matrices and LQR gain if not already set
if isempty(K)
    % Quadrotor Parameters

```

```

    g = params.grav;    % Gravity
    m = params.mass;    % Mass
    invI = params.invI; % Inverse inertia matrix

    % State-space matrices
    %theta and phi = 0
    r_ang=0;
    p = 0;
    A = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0;
         0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0;
         0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0;
         0, 0, 0, 0, 0, 0, 0, g, 0, 0, 0, 0;
         0, 0, 0, 0, 0, 0, -g, 0, 0, 0, 0, 0;
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
         0, 0, 0, 0, 0, 0, 0, cos(0)*r_ang - sin(0)*p, 0, cos(0), 0, sin(0);
         0, 0, 0, 0, 0, 0, ((sec(0))^2)*(sin(0)*p - cos(0)*r_ang), tan(0)*(sin(0)*r
         0, 0, 0, 0, 0, 0, tan(0)*sec(0)*(-p*sin(0) + r_ang*cos(0)), -(cos(0)*p + s
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0;
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0];

    B = [zeros(5, 4);
         [1/m, 0, 0, 0];
         zeros(3, 4);
         zeros(3, 1), invI];

    % Cost matrices
    Q = diag([1000, 1000, 50, 25, 25, 0.01, 10, 10, 1, 0.1, 0.1, 0.1]);
    R = diag([0.01, 1, 1, 1]);
    % these values are what i got experimentally, it tracks with a great
    % precision
    % Compute LQR gain
    K = lqr(A, B, Q, R);
end

% Current States
pos = qd{qn}.pos;    % Position [x, y, z]
vel = qd{qn}.vel;    % Velocity [vx, vy, vz]
euler = qd{qn}.euler; % [Roll, Pitch, Yaw]
omega = qd{qn}.omega; % [p, q, r]

```

```

% Desired States
desired_state = trajhandle(t, qn);
pos_des = desired_state.pos;
vel_des = desired_state.vel;
yaw_des = desired_state.yaw;
yawdot_des = desired_state.yawdot;

% Define the state and desired state
X = [pos; vel; euler; omega];
X_des = [pos_des; vel_des; [0; 0; yaw_des]; [0; 0; yawdot_des]];

% Error in States
E_x = X - X_des;

% Control Input
u = -K * E_x;

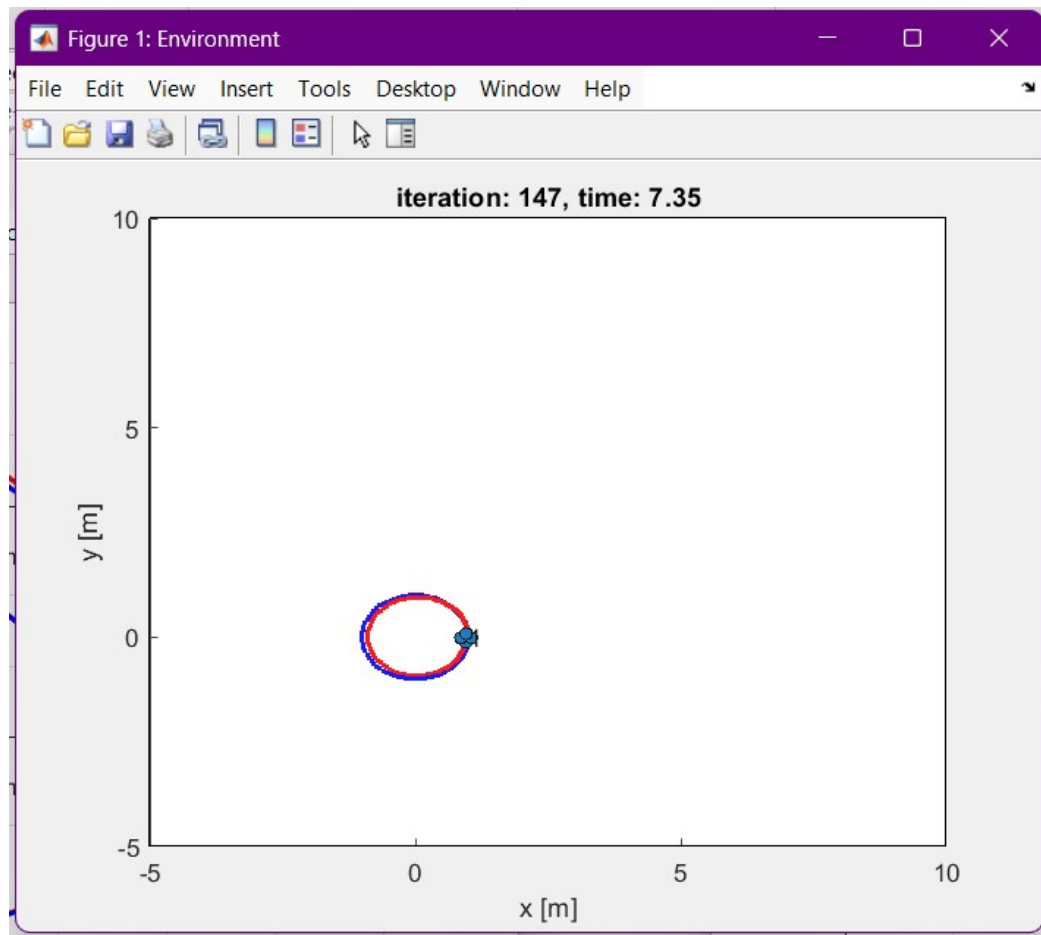
% Extract thrust and moments
F = u(1);          % Thrust
M = u(2:4);        % Moments [Mx, My, Mz]

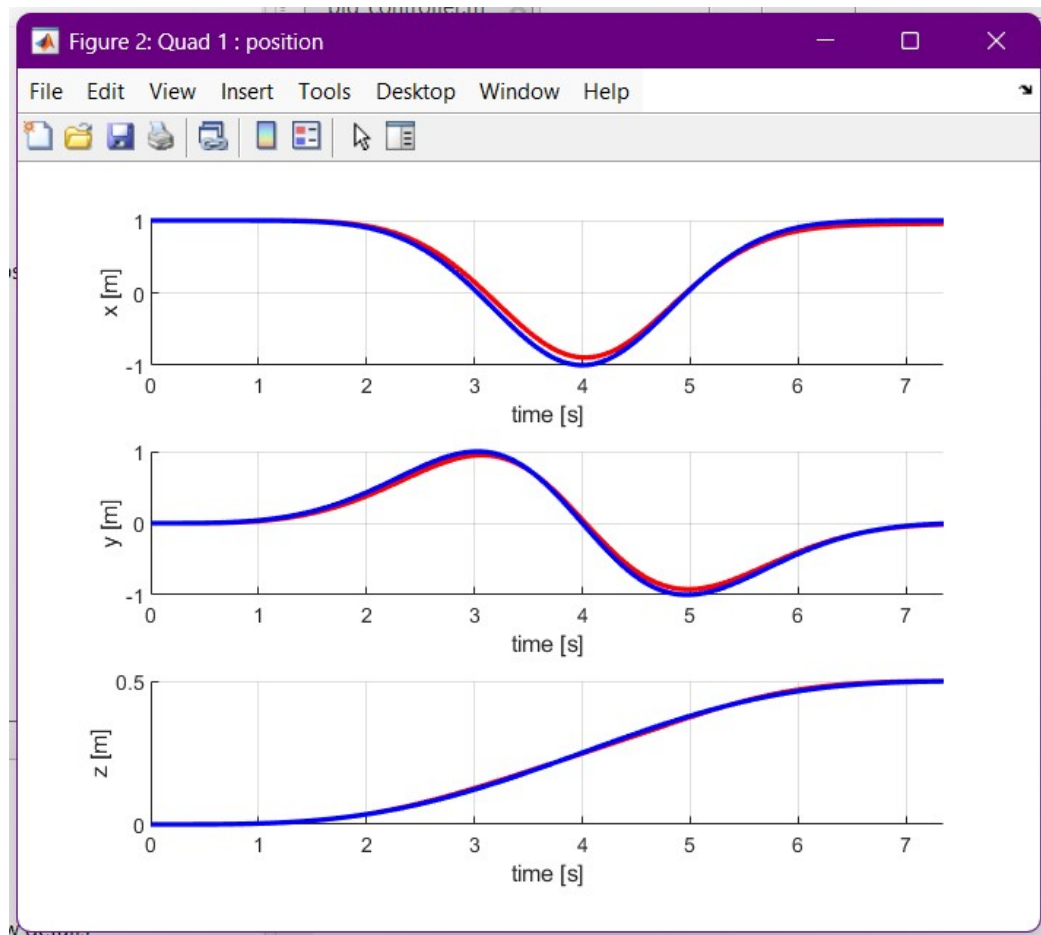
% Output trpy and drpy for hardware
trpy = [0, 0, 0, F]; % Thrust and desired roll, pitch, yaw
drpy = [0, 0, 0, 0]; % Derivatives (not used here)
end

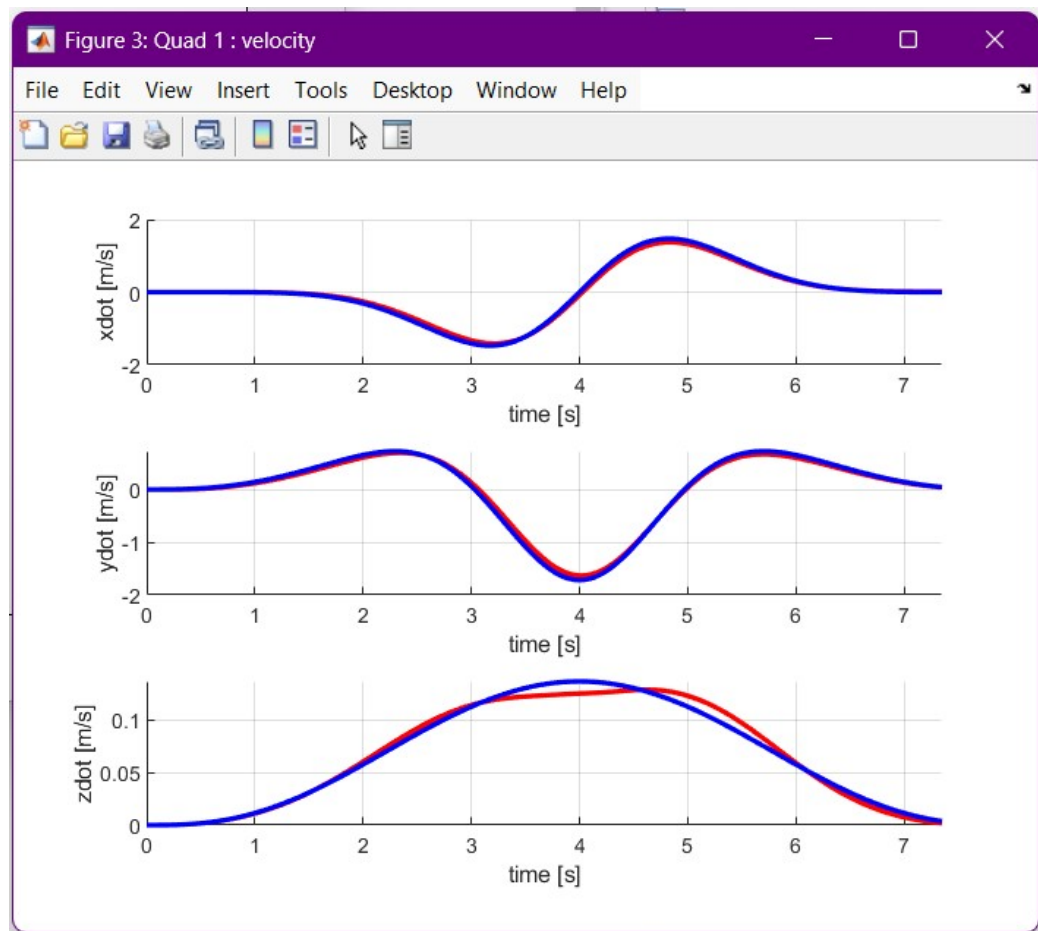
```

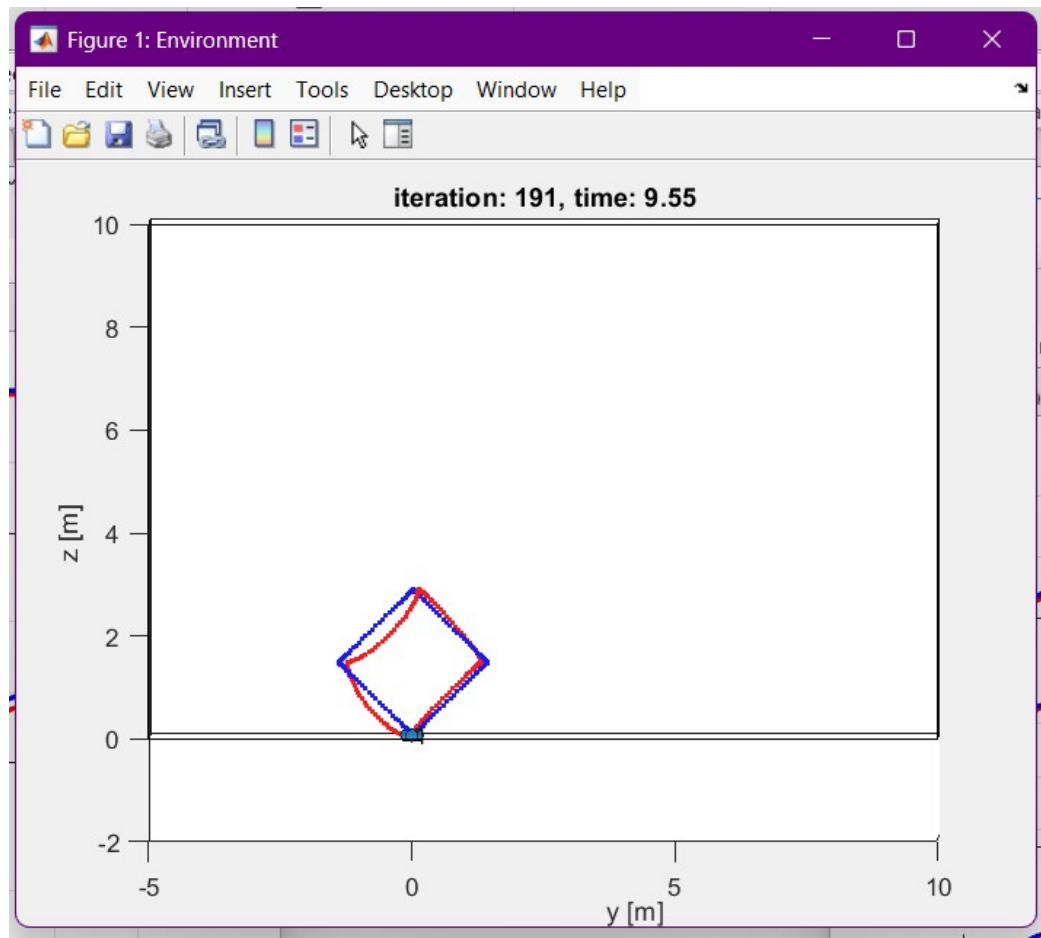
5 Observation

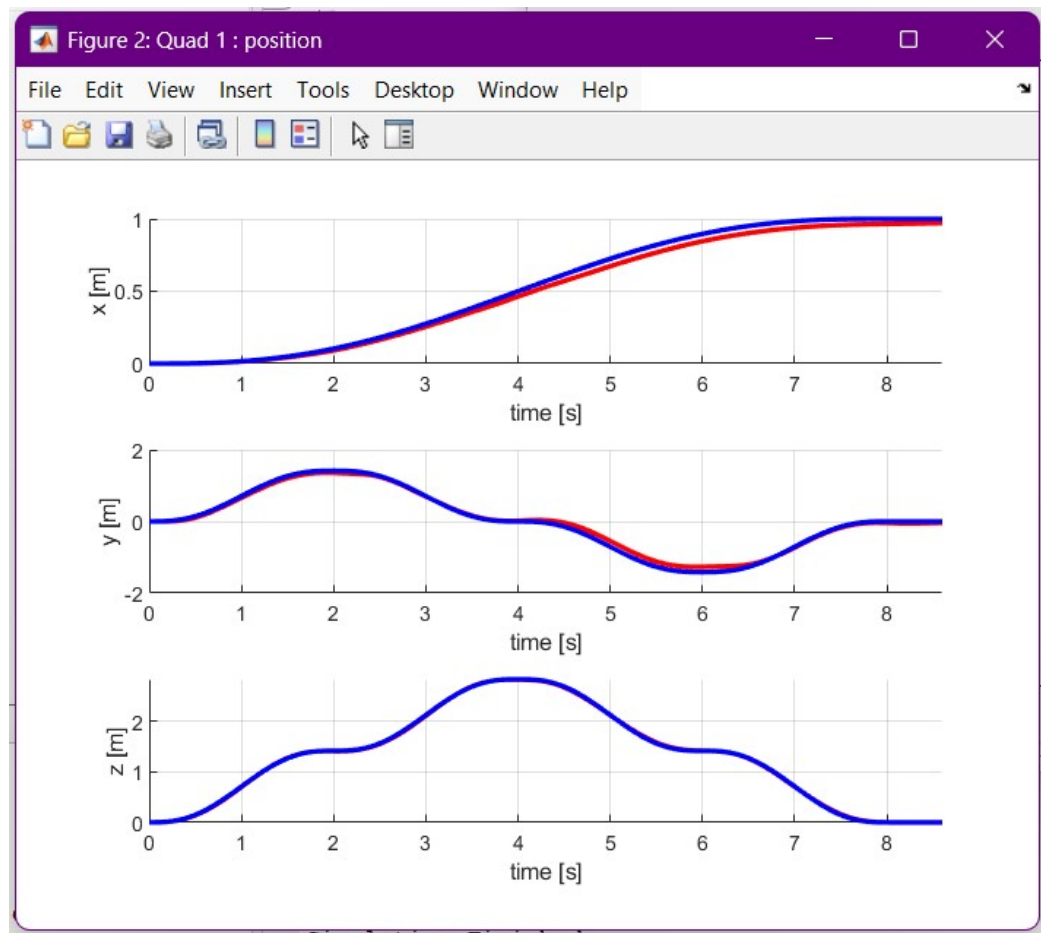
The PD controller provided satisfactory tracking performance for simple trajectories. However, it struggled with aggressive maneuvers due to its reliance on error feedback and since we were working with Hover conditions. moreover it was always off by some amount. Specially in the diamond shape, it was very notable when it started descending. Trying to fix the gains for one trajectory often led to a bad impact on the other trajectory which led to complications.

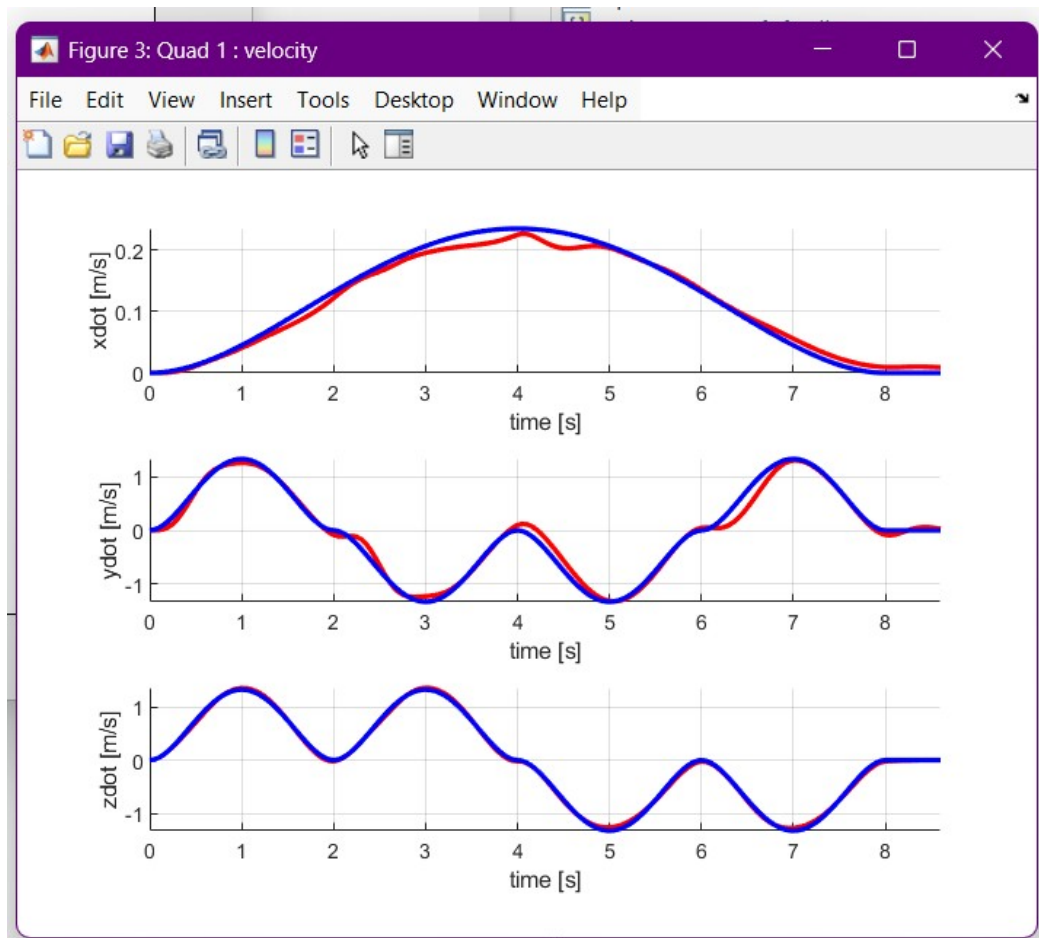




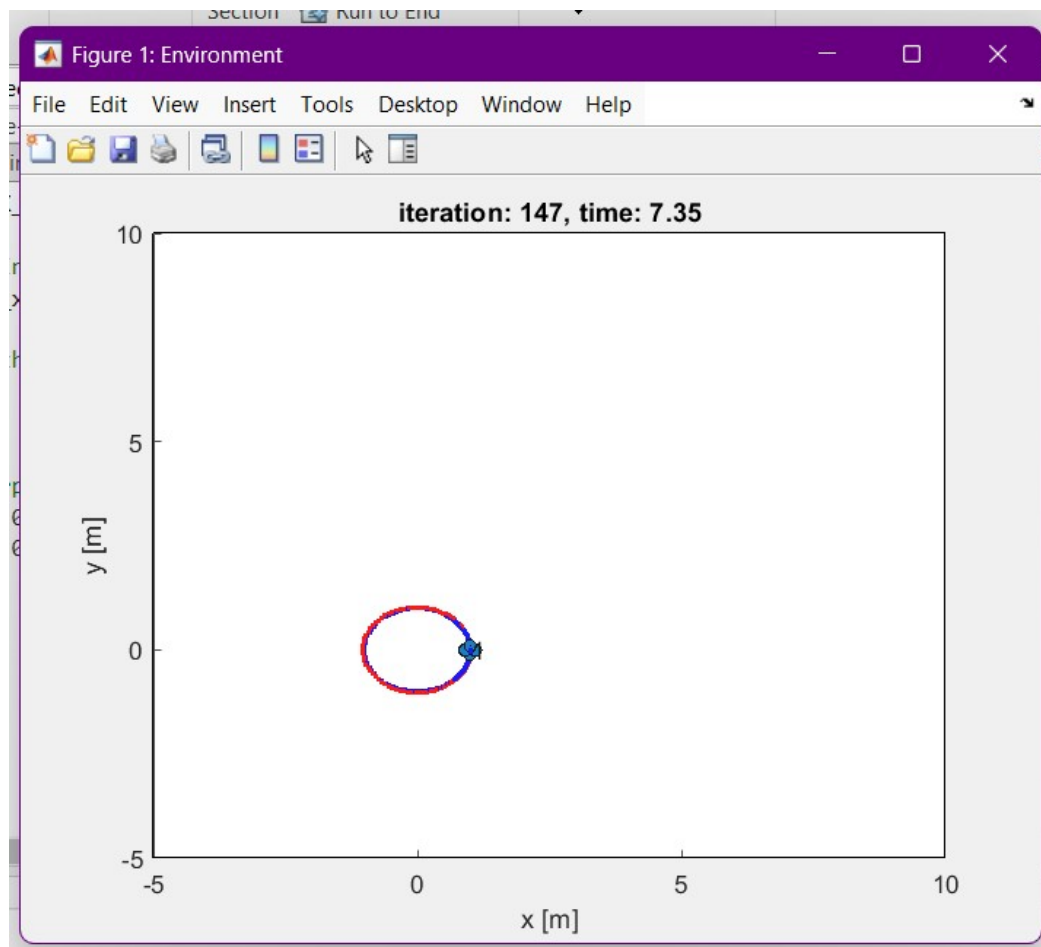


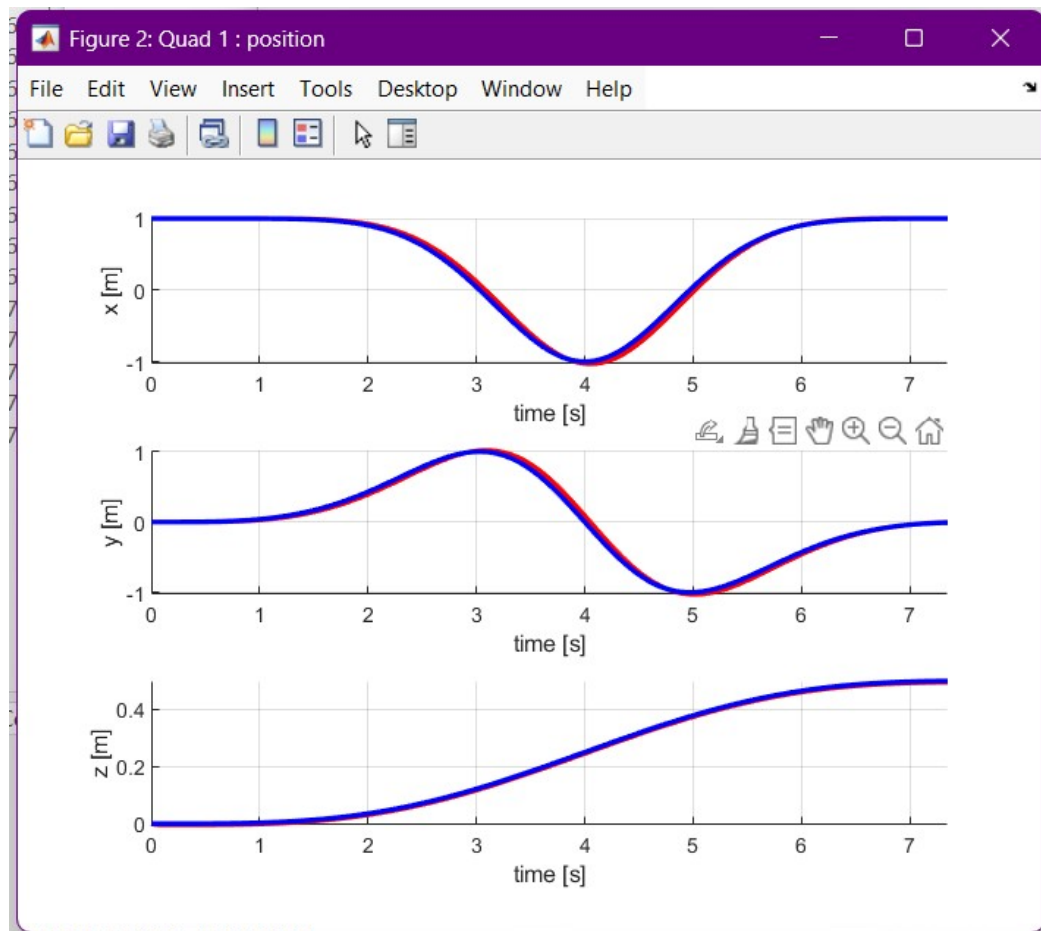


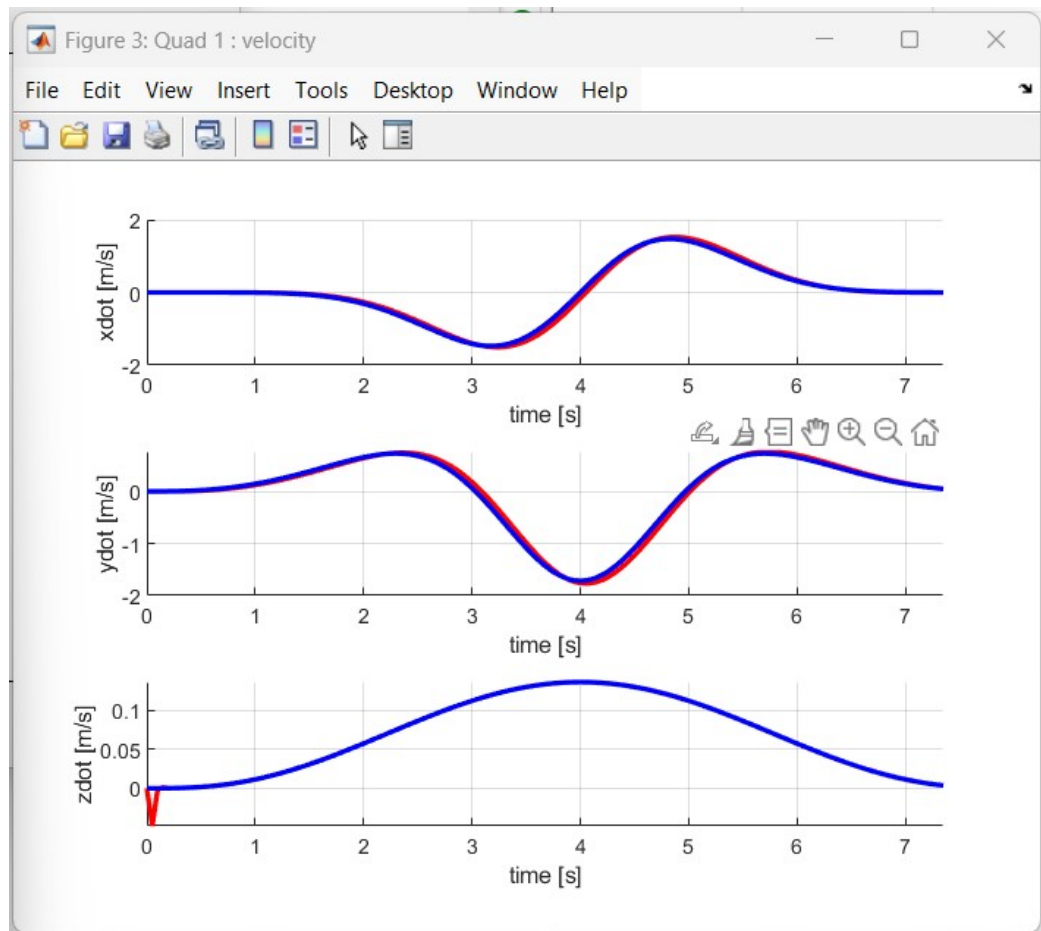


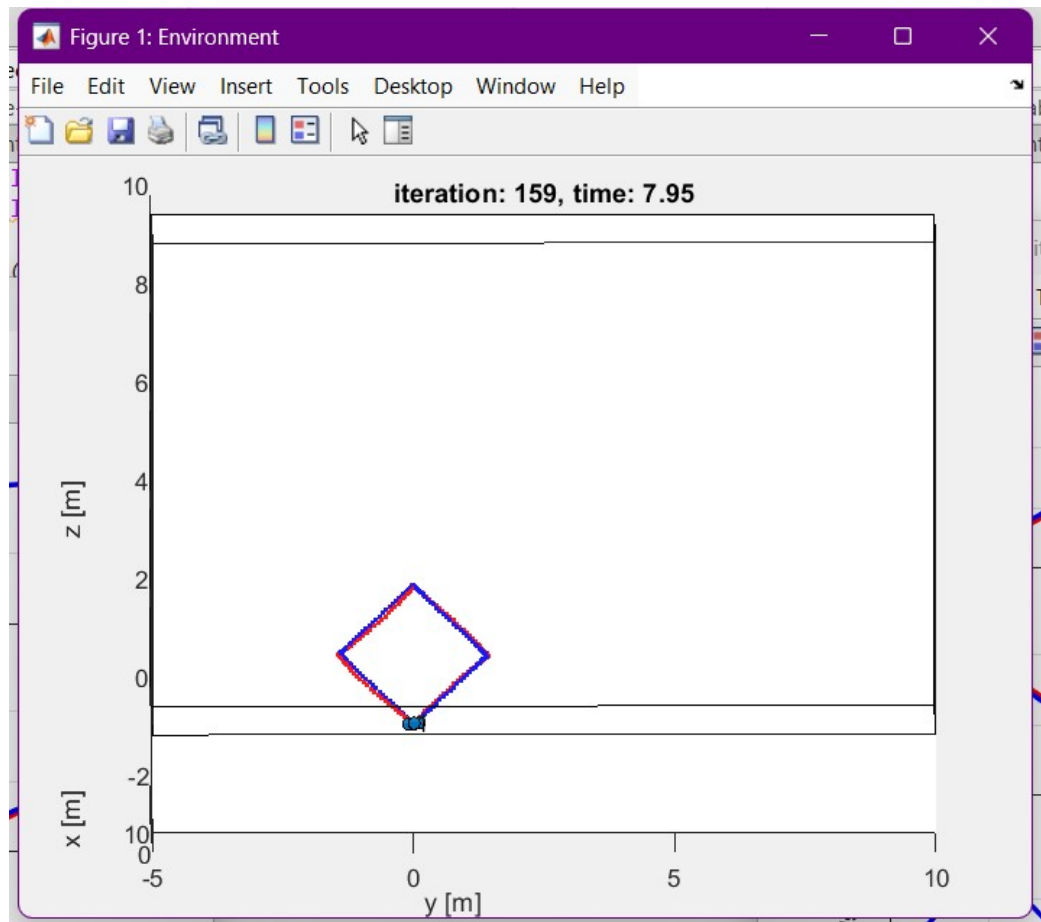


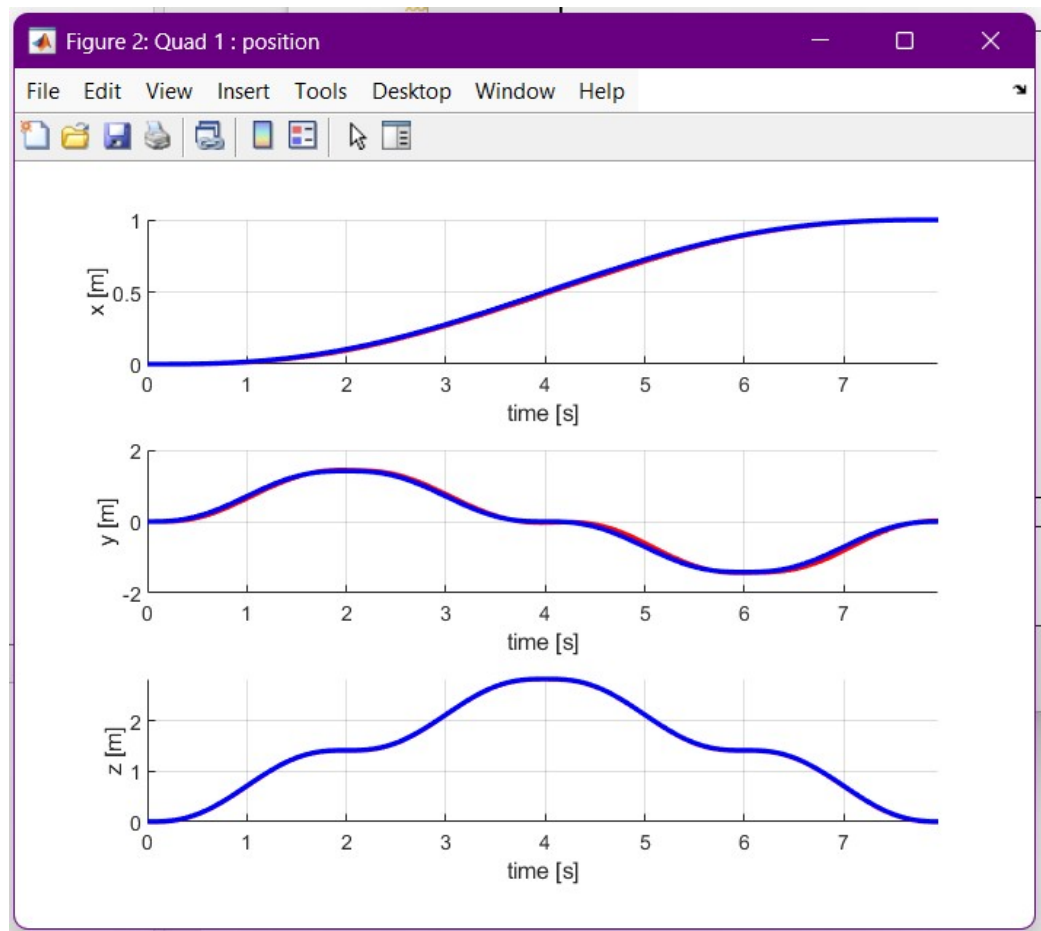
The **LQR controller** demonstrated superior performance, particularly for complex trajectories, due to its optimal control design. here I was able to specifically fix the weights for x and y position and velocities as they seemed to be needing higher weights for optimal tracking. and unlike PD controller, the same weights worked just as right for the other trajectory as well.

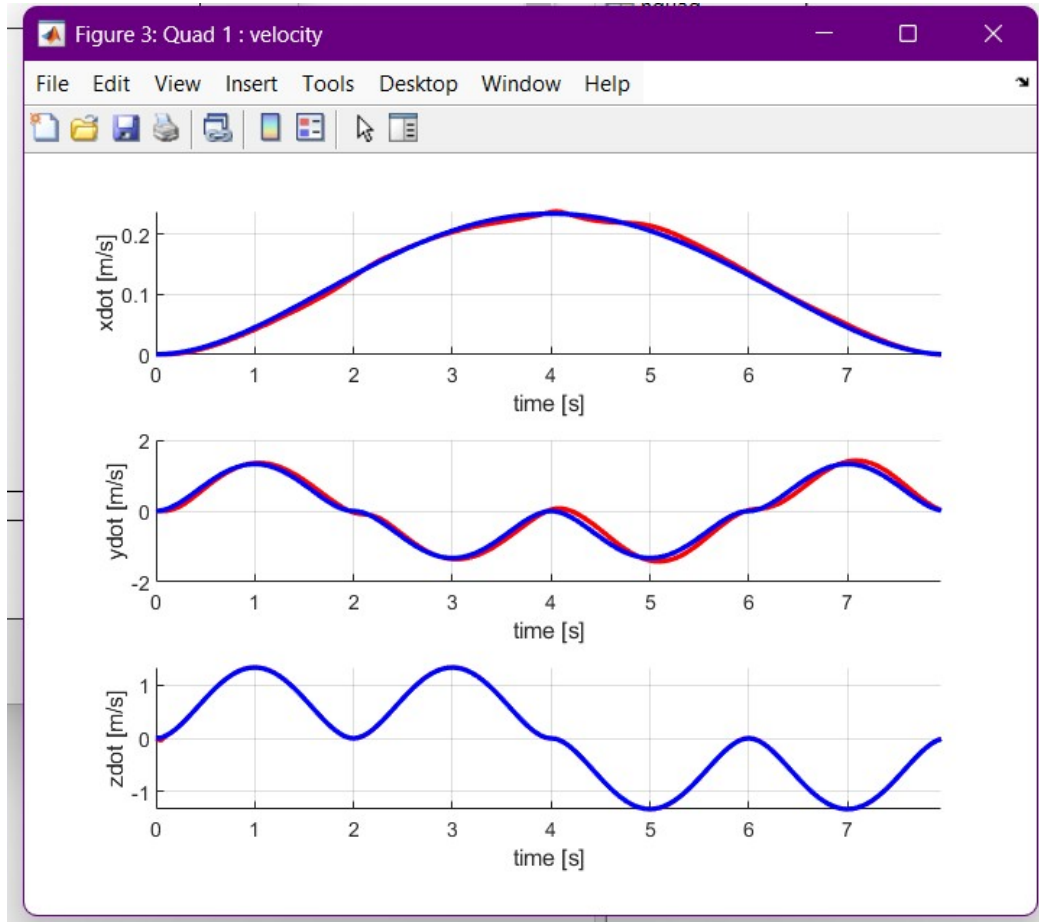












6 Conclusion

The project successfully implemented and compared PD and LQR controllers for quadrotor - CrazyFlie2.0 's trajectory tracking. The LQR controller outperformed the PD controller in terms of tracking accuracy and stability, making it more suitable for dynamic and complex tasks.

If we compare the time and accuracy it took to complete the task, we can see, for the Simpler tracking trajectory : Circle although my PD and LQR took the same amount of time, the LQR had a superior tracking in terms of stability.

Whereas In Diamond we can observe a major difference, it took about 9.55 s for PD Controller to complete the shape while causing some issue in descent, whereas LQR was able to complete the trajectory not only faster, in about 7.95 s but with higher accuracy as well.